

COMSATS University Islamabad

Attock Campus



Department Of Computer Science

<i>Course:</i>	<i>Information Security</i>
<i>Instructor:</i>	<i>Ms. Ambreen Gul</i>
<i>L_Assignment</i>	<i>03</i>
<i>Due Date:</i>	<i>4th May, 2026</i>

Student Details

<i>Registration No:</i>	<i>FA24-BSE-030</i>
<i>Name:</i>	<i>Sayed Aliza Hashmi</i>

Task 1-- Blockchain:

Task 1: Basic Blockchain Implementation

1. Code Implementation – Line-by-Line Explanation

Imports

```
import hashlib  
  
import time  
  
import random
```

hashlib provides the SHA-256 hash function used to secure each block.

time is used to record a timestamp when a block is created.

random is imported but only used later in the ECC task; it could be omitted for Task 1.

Class Block

```
class Block:  
  
    def __init__(self, index, data, previous_hash):  
  
        self.index = index  
  
        self.timestamp = time.time()  
  
        self.data = data  
  
        self.previous_hash = previous_hash  
  
        self.hash = self.compute_hash()
```

`__init__`: The constructor initialises a block with an index (block number), transaction data, and the hash of the previous block

self.timestamp stores the current system time (seconds since epoch).

self.hash immediately calculates the block's own cryptographic fingerprint.

```
def compute_hash(self):  
    block_content = f"{self.index}{self.timestamp}{self.data}{self.previous_hash}"  
    return hashlib.sha256(block_content.encode()).hexdigest()
```

compute_hash: Concatenates the block's fields into one string, encodes it to bytes, and returns the SHA-256 hex digest. This binds everything together—if any field changes, the hash changes completely.

```
def __repr__(self):  
    return (f"Block(Index: {self.index}, Timestamp: {self.timestamp:.2f}, "  
           f"Data: {self.data}, PrevHash: {self.previous_hash[:8]}..., "  
           f"Hash: {self.hash[:8]}...)"
```

__repr__: Provides a readable string representation for printing. Only the first 8 characters of the hashes are shown for brevity.

Class Blockchain

```
class Blockchain:  
    def __init__(self):  
        self.chain = []  
        self.create_genesis_block()
```

`__init__`: Initialises an empty list chain and immediately creates the genesis block.

```
def create_genesis_block(self):  
  
    genesis = Block(0, "Genesis Block", "0" * 64)  
  
    self.chain.append(genesis)
```

`create_genesis_block`: Creates the first block (index 0) with a dummy previous hash (64 zeros). This block has no actual predecessor.

```
def add_block(self, data):  
  
    last_block = self.chain[-1]  
  
    new_block = Block(last_block.index + 1, data, last_block.hash)  
  
    self.chain.append(new_block)
```

`add_block`: Takes new transaction data, gets the last block, increments its index, and creates a new block whose `previous_hash` is the hash of the last block. It then appends the new block to the chain.

```
def is_chain_valid(self):  
  
    for i in range(1, len(self.chain)):  
  
        current = self.chain[i]  
  
        previous = self.chain[i - 1]  
  
        if current.hash != current.compute_hash():  
  
            print(f"Tampering detected: Block {current.index} hash is invalid.")
```

```
        return False

    if current.previous_hash != previous.hash:

        print(f"Tampering detected: Block {current.index} previous_hash mismatch.")

        return False

    return True
```

is_chain_valid: Loops through every block (except the genesis).

Recomputes the block's hash and compares it with the stored hash. If different, data has been modified.

Checks that the block's previous_hash matches the hash of the immediately preceding block. If not, the link is broken.

Returns True only if all checks pass.

```
def display_chain(self):

    for block in self.chain:

        print(block)
```

display_chain: Prints each block using the __repr__ method.

Main Execution (Task 1)

```
if __name__ == "__main__":

    print("=== Task 1: Basic Blockchain ===")
```

```
bc = Blockchain()

bc.add_block("Alice pays Bob 10 coins")

bc.add_block("Bob pays Charlie 5 coins")

bc.add_block("Charlie pays Dave 2 coins")

bc.add_block("Dave pays Alice 1 coin")

bc.display_chain()
```

Creates a new blockchain, adds four transaction blocks, and prints all blocks.

```
print("\nBlockchain valid?", bc.is_chain_valid())
```

Validates the chain immediately after creation – should print True.

```
print("\n--- Tampering block 1's data ---")

bc.chain[1].data = "Alice pays Bob 1000 coins"

print("Blockchain valid after tampering?", bc.is_chain_valid())
```

Changes the data of block 1 without recomputing its hash. The validation fails, proving that tampering is detected.

2. Security Analysis

Immutability: Each block contains the hash of the previous block. Changing data in any block alters its hash, which in turn breaks the link to the next block because the next block's `previous_hash` no longer matches. To successfully tamper with one block, an attacker would need to recompute the hashes of that block and all subsequent blocks, which becomes computationally infeasible in a longer chain, especially when combined with a consensus mechanism (e.g., proof-of-work).

cryptographic Hash Strength: SHA-256 provides strong preimage and collision resistance. It is infeasible to find two different inputs that produce the same hash, or to reverse a hash back to its original input. Thus a block's integrity is assured.

Chain Integrity: The validation function checks both the internal hash correctness and the inter-block linkage. Any mismatch is immediately flagged.

Limitations: This implementation is a centralised, in-memory chain without a peer-to-peer network, consensus algorithm, or difficulty adjustment.

Task 2: Elliptic Curve Cryptography (ECC) Basics

Code Implementation – Line-by-Line Explanation

Class ECC

```
class ECC:
```

```
    def __init__(self, a, b, p):
```

```
        self.a = a
```

```
        self.b = b
```

```
        self.p = p
```

```
        if (4 * a**3 + 27 * b**2) % p == 0:
```

```
            raise ValueError("Curve is singular, choose different parameters.")
```

`__init__`: Stores curve parameters a , b , and prime modulus p . Checks the discriminant $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$ to ensure the curve is non-singular (no cusps or self-intersections). A singular curve is insecure.

```
    def is_on_curve(self, x, y):
```

```
return (y**2) % self.p == (x**3 + self.a * x + self.b) % self.p
```

`is_on_curve`: Verifies that the point (x, y) satisfies the curve equation $y^2 \equiv x^3 + ax + b \pmod{p}$.

```
def mod_inverse(self, k):  
    if k % self.p == 0:  
        raise ZeroDivisionError("Inverse does not exist.")  
    return pow(k, -1, self.p)
```

`mod_inverse`: Computes the modular inverse of k modulo p . Uses Python's built-in `pow` with a negative exponent (available from Python 3.8), which performs the Extended Euclidean algorithm internally.

Python

```
def point_add(self, P, Q):  
    if P is None:  
        return Q  
    if Q is None:  
        return P  
    x1, y1 = P  
    x2, y2 = Q  
    if x1 == x2 and y1 != y2:  
        return None
```

```

if P != Q:

    m = ((y2 - y1) * self.mod_inverse(x2 - x1)) % self.p

else:

    if y1 == 0:

        return None

    m = ((3 * x1**2 + self.a) * self.mod_inverse(2 * y1)) % self.p

x3 = (m**2 - x1 - x2) % self.p

y3 = (m * (x1 - x3) - y1) % self.p

return (x3, y3)

```

point_add: Implements the group law for elliptic curves.

If one point is the identity (None), return the other

If $x_1 == x_2$ but $y_1 \neq y_2$, the points are inverses $\rightarrow$ return None (point at infinity).

For distinct points, slope $m = (y_2 - y_1) / (x_2 - x_1)$.

For a doubling ($P == Q$), slope $m = (3x_1^2 + a) / (2y_1)$. If $y_1 == 0$, the tangent is vertical, so the result is the point at infinity.

Coordinates of the sum are calculated using standard formulas modulo p .

```

def scalar_mult(self, k, P):

    if P is None:

        return None

    result = None

    addend = P

    while k:

```

```

    if k & 1:

        result = self.point_add(result, addend)

    addend = self.point_add(addend, addend)

    k >>= 1

    return result

```

scalar_mult: Performs scalar multiplication $k \cdot P$ using the double-and-add algorithm (analogous to exponentiation by squaring). It processes the binary representation of k and is efficient with $O(\log k)$ point additions.

```

def find_point_from_x(self, x):

    rhs = (x**3 + self.a * x + self.b) % self.p

    for y in range(self.p):

        if (y * y) % self.p == rhs:

            return (x, y)

    return None

```

```

find_point_from_x(self, x):

    rhs = (x**3 + self.a * x + self.b) % self.p

    for y in range(self.p):

        if (y * y) % self.p == rhs:

            return (x, y)

    return None

```

`find_point_from_x`: Given an x-coordinate, computes the right-hand side of the curve equation and brute-forces all possible y in $[0, p-1]$ to find a matching square. This is only feasible for very small p (here 17). In practice, Tonelli-Shanks would be used.

```
def encode_message(self, m):  
  
    x = m  
  
    while x < self.p:  
  
        point = self.find_point_from_x(x)  
  
        if point:  
  
            return point  
  
        x += 1  
  
    raise ValueError("Could not encode message – increase message space.")
```

`encode_message`: Converts an integer message m into a point on the curve. It tries $x = m, m+1, \dots$ until a valid y exists. This is a simple probabilistic embedding; the probability of failure is low if p is much larger than m (but here we use small numbers).

`ecc_demo()` Function

```
def ecc_demo():  
  
    print("\nTASK 2: ECC Demonstration")  
  
    curve = ECC(a=0, b=7, p=17)  
  
    G = (1, 5)  
  
    print(f"Curve:  $y^2 = x^3 + 7 \pmod{17}$ , Generator G = {G}")
```

Chooses the curve $y^2 = x^3 + 7 \pmod{17}$ (the same equation as Bitcoin's secp256k1 but with a tiny prime). Sets generator point $G = (1,5)$, which is on the curve.

```
private_key = random.randint(2, curve.p - 1)

public_key = curve.scalar_mult(private_key, G)

print(f"Private key (d): {private_key}")

print(f"Public key (Q = d*G): {public_key}")
```

Generates a random private key d between 2 and 16 (since $p=17$). Computes the public key $Q = d \cdot G$ via scalar multiplication.

```
message = 9

M = curve.encode_message(message)

print(f"Original message (numeric): {message}")

print(f"Encoded message point M: {M}")
```

Takes the numeric message 9 and encodes it into a curve point M .

```
k = random.randint(2, curve.p - 1)

C1 = curve.scalar_mult(k, G)

shared_secret = curve.scalar_mult(k, public_key)

C2 = curve.point_add(M, shared_secret)

print(f"Ciphertext: C1 = {C1}, C2 = {C2}")
```

ElGamal encryption:

Chooses a random ephemeral key k .

Computes $C1 = k \cdot G$ and shared secret $k \cdot Q$.

$C2$ is the message point M plus the shared secret (point addition).

```
decrypt_shared = curve.scalar_mult(private_key, C1)

inv_shared = (decrypt_shared[0], (-decrypt_shared[1]) % curve.p)

M_decrypted = curve.point_add(C2, inv_shared)

recovered_message = M_decrypted[0]

print(f"Decrypted message point: {M_decrypted}")

print(f"Recovered numeric message: {recovered_message}")
```

Decryption:

The receiver computes the same shared secret using their private key: $d \cdot C1$.

To subtract the shared secret from $C2$, it calculates the inverse of the point (same x , negated y modulo p) and adds it to $C2$.

The x -coordinate of the resulting point is the original integer message.

Security Analysis – Task 2

Elliptic Curve Discrete Logarithm Problem (ECDLP): The security of ECC relies on the fact that, given G and $d \cdot G$, it is computationally infeasible to determine d . On properly chosen curves with large prime order, this problem is exponentially hard.

Curve Choice: The demo uses a curve over a tiny finite field ($p=17$) for educational purposes. This curve has very few points and is trivially broken with brute force. In practice, standardised curves like `secp256k1` or `Curve25519` are used, with primes of at least 256 bits, making ECDLP attacks impossible with current technology.

Encryption Scheme: The simplified EC-ElGamal is provably secure under the Computational Diffie-Hellman (CDH) assumption. However, it is not IND-CCA secure on its own; in real systems, an authenticated encryption mode (ECIES) is used to prevent chosen-ciphertext attacks.

Randomness: The ephemeral key k must be truly random and never reused. Reusing k for two different messages would expose the private key, as the two ciphertexts C_2 and C_2' would allow an attacker to compute the difference and correlate messages.

Message Encoding: The encoding method (trying successive x-coordinates) is simple but deterministic and could leak information about the plaintext length. In a real implementation, Koblitz's method or Elligator maps are used to provide indistinguishability from random points.

Point Operations: The addition and doubling formulas are correct and handle edge cases (points at infinity, doubling when $y=0$) to avoid exceptions that could be exploited in side-channel attacks. However, this implementation does not use constant-time operations, so it would be vulnerable to timing attacks if used in a production environment.

```
PS C:\Users\Laptop-Zone-BY-JK\Documents\Information_Security_A3> & C:/Users/Laptop-Zone-BY-JK/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Laptop-Zone-BY-JK/Documents/Information_Security_A3/Q1.py
PS C:\Users\Laptop-Zone-BY-JK\Documents\Information_Security_A3> random
2067036772
PS C:\Users\Laptop-Zone-BY-JK\Documents\Information_Security_A3> & C:/Users/Laptop-Zone-BY-JK/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Laptop-Zone-BY-JK/Documents/Information_Security_A3/Q2.py
=== Task 1: Basic Blockchain ===
Block(Index: 0, Timestamp: 1777871428.83, Data: Genesis Block, PrevHash: 00000000..., Hash: b6ce66e0...)
Block(Index: 1, Timestamp: 1777871428.83, Data: Alice pays Bob 10 coins, PrevHash: b6ce66e0..., Hash: e96f2c2d...)
Block(Index: 2, Timestamp: 1777871428.83, Data: Bob pays Charlie 5 coins, PrevHash: e96f2c2d..., Hash: 6ed0eb40...)
Block(Index: 3, Timestamp: 1777871428.83, Data: Charlie pays Dave 2 coins, PrevHash: 6ed0eb40..., Hash: b97a5549...)
Block(Index: 4, Timestamp: 1777871428.83, Data: Dave pays Alice 1 coin, PrevHash: b97a5549..., Hash: 1b8b00f1...)

Blockchain valid? True

--- Tampering block 1's data ---
Tampering detected: Block 1 hash is invalid.
Blockchain valid after tampering? False
```

```
TASK 2: ECC Demonstration
Curve:  $y^2 = x^3 + 7 \pmod{17}$ , Generator  $G = (1, 5)$ 
Private key (d): 10
Public key ( $Q = d \cdot G$ ): (1, 5)

Original message (numeric): 9
Encoded message point M: (10, 2)
Ciphertext:  $C1 = (5, 8)$ ,  $C2 = (15, 4)$ 
Decrypted message point: (10, 2)
Recovered numeric message: 10
PS C:\Users\Laptop-Zone-BY-JK\Documents\Information_Security_A3> 
```